Arenula

MCP server suite for the s&box game engine

19	~120	3	8-13K
TOOLS	ACTIONS	SERVERS	TOKENS

Three Servers, One Suite

Each server has a distinct role. The Editor runs inside s&box; the API and Docs servers run standalone.

Editor

Scene manipulation, code compilation, asset management, terrain editing. Runs inside the s&box editor as a C# library plugin.

19 tools SSE :8098 C#

API

Offline type and member reference. Indexes ~1,800 types and ~15,000 members from the s&box CDN schema.

8 tools stdio TypeScript

Docs

Narrative documentation fetcher. Scrapes docs.facepunch.com, converts to Markdown, caches with LRU.

3 tools stdio TypeScript

Editor Tools

19 omnibus tools. Each takes a required `action` parameter to select the operation.

`scene`	`summary` `hierarchy` `statistics` `find` `find_in_radius` `get_details` `prefab_instances`	7
`gameobject`	`create` `destroy` `duplicate` `reparent` `rename` `enable` `set_tags` `set_transform` `batch_transform`	9

component	add · remove · set_property · set_enabled · get_properties · get_types · copy	7
compile	trigger · status · errors · generate_solution · wait	5
prefab	instantiate · get_structure · get_instances · break · update · create · save_overrides · revert · get_overrides	9
asset_query	browse · search · open · get_dependencies · get_model_info · get_material_properties · get_mesh_info · get_bounds · get_unsaved · get_status · get_json · get_references	12
asset_manage	create · delete · rename · move · save · reload · get_references	7
editor	select · get_selected · set_selected · clear_selection · frame_selection · get_play_state · start_play · stop_play · get_log · save_scene · save_scene_as · undo · redo · console_list · console_run · open_code_file · get_preferences · set_preference	18
session	list · set_active · load_scene	3
lighting	create · configure · create_skybox · set_skybox	4
physics	add_collider · configure_collider · add_rigidbody · create_model_physics · create_character_controller · create_joint	6
audio	create · configure	2
effects	create · configure_particle · configure_post_processing	3
camera	create · configure	2
mesh	create_block · create_clutter · set_face_material · set_texture_params · set_vertex_position · set_vertex_color · set_vertex_blend · get_info	8
navmesh	create_agent · create_area · create_link · generate · get_status · query_path	6
cloud	search · get_package · mount	3
project	get_collision · set_collision_rule · get_input · get_info	4
terrain	create · configure · get_info · paint_material · sync	5

Architecture

28 C# source files organized into Core infrastructure and per-tool Handlers.

```
AI Client
    |
  HTTP / SSE
    |
ArenulaMcpServer
    |
RpcDispatcher
    |
    +-------------------+-------------------+
    |                   |                   |
Async Path          Console Path        Main Thread
compile, cloud      exception safe      all other tools
    |                   |                   |
 Handler             Handler             Handler
(background)      (sync isolation)   (GameTask.MainThread)
```

Core (8 files)

`ArenulaMcpServer` HTTP/SSE listener and session management. `RpcDispatcher` extracts tool name + action, routes to the correct handler. `ToolRegistry` holds all 19 tool schemas. `HandlerBase` provides response formatting, pagination, and error helpers. `SceneHelpers` handles scene resolution and tree walking.

Handlers (20 files)

One file per tool. Each exports a `Handle(action, args)` method with an internal switch on the action enum. Async handlers (`CompileHandler` , `CloudHandler`) use `HandleAsync` and dispatch before `GameTask.MainThread()` to prevent deadlocks.

Threading Model

Scene APIs must run on the main thread. The dispatcher calls `await GameTask.MainThread()` before invoking sync handlers. Async handlers (network/compile) run on the background thread and may internally switch to main thread for specific sub-operations.

Design Principles

Following [Anthropic's MCP tool design guidance](#).

Omnibus Tools

Related operations grouped with action enum parameter. Fewer tools means less selection ambiguity for the LLM.

Rich Descriptions

3-4 sentences per tool explaining what it does, when to use it, and when not to. Negative guidance prevents misuse.

Trimmed Responses

Canonical {id, name} tuples. Pagination with offset/limit. 16K character truncation. Concise and detailed modes.

Actionable Errors

Structured errors with action context and suggestions. Fuzzy type matching: "Did you mean Rigidbody?"

Schema Safety

additionalProperties: false on every schema. Enum constraints on actions and sub-types. Prevents hallucination.

Async Safety

Network tools dispatch before MainThread to avoid deadlocks. Never block the main thread with synchronous waits.

Installation

1. Copy the Editor plugin

```
# Copy into your s&box project's Libraries folder
Arenula-MCP/editor/  ⟶  YourProject/Libraries/arenula_mcp/
```

2. Build the API and Docs servers

```
# From the Arenula-MCP directory
cd api && npm install && npm run build
cd ../docs && npm install && npm run build
```

3. Configure .mcp.json in your project root

```
{
  "mcpServers": {
    "editor": { "type": "sse", "url": "http://localhost:8098/sse" },
    "api":    { "command": "node", "args": ["path/to/Arenula-MCP/api/dist/index.js"] },
    "docs":   { "command": "node", "args": ["path/to/Arenula-MCP/docs/dist/index.js"] }
  }
}
```

4. Open s&box

Arenula Editor compiles automatically and starts the MCP server on port 8098.

Based on Ozmium MCP Server by ozmium7 · API from sbox-api-mcp by sofianebel · Docs from sbox-docs-mcp
GPL-3.0 (Editor) · MIT (API, Docs)